

**Politecnico di Milano**  
School of Industrial and Information Engineering

**Jan Wangrat**

# **Intelligent Job Management**

**Final Report**  
**COMPUTER SCIENCE AND ENGINEERING**

Supervisor:  
**Prof. Danilo Ardagna**  
Politecnico di Milano  
with Federica Filippini, Ph.D. Candidate

Milano, June 29, 2026

## **Abstract**

Intelligent Job Management (IJM) is a scheduler for shared GPU clusters that combines profile-aware placement, deadline-driven preemption, and cross-version checkpoint resume. It extends the ANDREAS project (Politecnico di Milano) from a single-host prototype to a distributed control plane: one FastAPI service drives per-node workers over SSH-tunnelled HTTP, an external GPUspb optimiser handles standard-run placement, and training jobs migrate between heterogeneous GPU generations (A40, QuadroP600) without losing progress. The report describes the deployment topology, the four-phase scheduling pipeline, the configuration surface, and two end-to-end scenarios that exercise the system on a two-node cluster.

## **Keywords**

GPU scheduling, deep learning, job management, profile-aware optimisation, distributed systems, FastAPI, PostgreSQL, Docker

# Contents

|                                                            |    |
|------------------------------------------------------------|----|
| <b>1. Introduction</b>                                     | 2  |
| <b>2. System Architecture</b>                              | 3  |
| 2.1. Deployment topology                                   | 3  |
| 2.2. Scheduling pipeline                                   | 5  |
| 2.3. Worker execution lifecycle                            | 9  |
| 2.4. State machine                                         | 11 |
| 2.5. Persistent state                                      | 12 |
| 2.6. Concurrency invariants                                | 13 |
| 2.7. Stoppable, resumable, cross-node training             | 13 |
| <b>3. Configuration and Job Submission</b>                 | 15 |
| 3.1. Cluster configuration                                 | 15 |
| 3.1.1. Nodes ( <code>nodes_config.json</code> )            | 15 |
| 3.1.2. Energy costs ( <code>gpu_energy_costs.json</code> ) | 16 |
| 3.2. Job submission                                        | 16 |
| 3.2.1. Request body                                        | 16 |
| 3.2.2. Response (201 Created)                              | 17 |
| 3.3. Full API overview                                     | 18 |
| 3.4. Runtime environment                                   | 18 |
| <b>4. End-to-End Scenarios</b>                             | 20 |
| 4.1. Objective and cost model                              | 20 |
| 4.1.1. Judging the proxy: an idealised yardstick           | 21 |
| 4.2. Available job types                                   | 22 |
| 4.3. Scenario 1 — single type                              | 22 |
| 4.3.1. Decision timeline                                   | 25 |
| 4.3.2. URGENT plan comparison                              | 25 |
| 4.4. Scenario 2 — two types, forced 2-GPU placement        | 27 |
| 4.4.1. URGENT plan comparison                              | 28 |
| 4.4.2. Decision timeline                                   | 29 |
| 4.5. Reproducibility                                       | 29 |
| <b>5. Conclusion</b>                                       | 30 |
| 5.1. Summary of contributions                              | 30 |
| 5.2. Limitations                                           | 30 |
| 5.3. Future work                                           | 30 |
| <b>Bibliography</b>                                        | 31 |

# 1. Introduction

Modern deep-learning research depends on shared GPU clusters in which training jobs compete for a small number of expensive accelerators. Operators are expected to place each job on the hardware that meets its deadline at the lowest cost, despite per-GPU prices that vary by more than an order of magnitude between generations. Manual scheduling quickly becomes the bottleneck, and off-the-shelf cluster managers (Kubernetes, plain SLURM) treat GPU bundles as interchangeable and have no notion of per-job energy budgets or deadline-aware preemption.

The ANDREAS project [AF<sup>+</sup>21] at Politecnico di Milano demonstrated that a cost-aware optimiser, fed with profile measurements of each job type on each hardware bundle, can plan placements that beat the typical operator’s intuition. ANDREAS, however, is a single-host prototype: it runs every container on the machine that hosts the API, has no remote worker, and assumes one homogeneous GPU pool. Real laboratories — the two-node MIM UW / Polimi cluster used in this work being a representative example — run jobs across nodes with different driver generations, must resume preempted training across CUDA versions, and want to consume a standalone optimiser as an HTTP service rather than ship it inside the scheduler.

This report describes Intelligent Job Management (IJM), a scheduler that extends the ANDREAS model to a distributed cluster. The control plane is a single FastAPI service that hosts the profiling scheduler, the optimiser client, and the per-node slot semaphores. Each GPU node runs a lightweight worker that owns the local Docker daemon. The two are glued together by plain HTTP over SSH tunnels, so the same code runs on a laptop talking to a remote cluster and on a single-host development machine. IJM integrates the GPUspb optimiser as the external decision maker for standard-run placement. Before any instance is placed as a standard run, the scheduler first executes a small number of profile runs for it on previously-unmeasured (node, GPU-count) cells, so the optimiser is never asked to score an unmeasured bundle. Checkpoints are written in a format that loads across torch versions, allowing a job to migrate between an A40 node and a legacy P600 node mid-training.

We validate the design with two end-to-end scenarios on the two-node cluster. Scenario 1 submits five `cnm_big` jobs to probe the profile sweep and the deadline-driven URGENT migration sequence on an under-subscribed cluster. Scenario 2 mixes two job types and fills every slot, pitting URGENT against priority-5 pins on A40 and slack lstm-small patients on P600. Together they separate two behaviours of the upstream GPUspb proxy: Scenario 1 exposes its *horizon-myopia* (an urgent job parked on slow but free hardware), while Scenario 2 shows that on a full cluster the right outcome still emerges from its stateless cost minimisation: the cheapest slack work is omitted from the plan (realised as a preemption) and, once an equal-priority pin finishes, the urgent job migrates onto the freed fast A40 and meets its deadline.

Chapter 2 describes the deployment topology, the scheduling pipeline, the state machine, the persistent state, and the invariants that protect the control plane from concurrency hazards. Chapter 3 documents the cluster and job configuration files, the submission API, and the runtime knobs. Chapter 4 presents the two scenarios and the cost model used by the optimiser. Chapter 5 summarises the contributions, the limitations, and the follow-up work that has concrete entry points in the codebase.

## 2. System Architecture

This section presents IJM’s deployment topology, the four-phase scheduling pipeline that drives every dispatch decision, the worker execution lifecycle that runs and stops each container, the job state machine, the persistent state held in PostgreSQL, and the concurrency invariants that keep the control plane consistent under racing API calls and worker notifications. Table 2.1 catalogues every component referenced in the rest of the chapter.

| Component          | Process                           | Responsibility                                                                                                        |
|--------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| API                | <code>ijm-api</code> (FastAPI)    | HTTP frontend; hosts the scheduling and dispatch loops                                                                |
| ProfilingScheduler | in-API                            | Picks the next unmeasured ( <code>node</code> , <code>GPU-count</code> ) cell for profile-owing rows                  |
| JobDispatcher      | in-API                            | Acquires the destination’s <code>NodeSlots</code> permit, then POSTs <code>/jobs/{id}/run</code> to the target worker |
| NodeSlots          | in-API                            | Per-node semaphore tracking GPU permits; reconciled with the DB by a periodic drift heartbeat                         |
| ClusterManager     | in-API                            | Loads <code>nodes_config.json</code> and exposes node and GPU resources                                               |
| Optimizer client   | in-API                            | Packages the GPUspb request, parses the returned assignments and preempts                                             |
| Worker             | <code>ijm-worker</code> (FastAPI) | Owns the local Docker daemon; performs atomic claim, runs the container, emits slot-freed notifications               |
| PostgreSQL         | DB                                | Source of truth for job rows and profile measurements                                                                 |
| GPUspb optimiser   | external HTTP                     | Cost-aware standard-run placement (reached via <code>OPTIMIZER_URL</code> )                                           |
| SPA                | static (Vite)                     | Operator UI; polls the API every 3–5 s                                                                                |

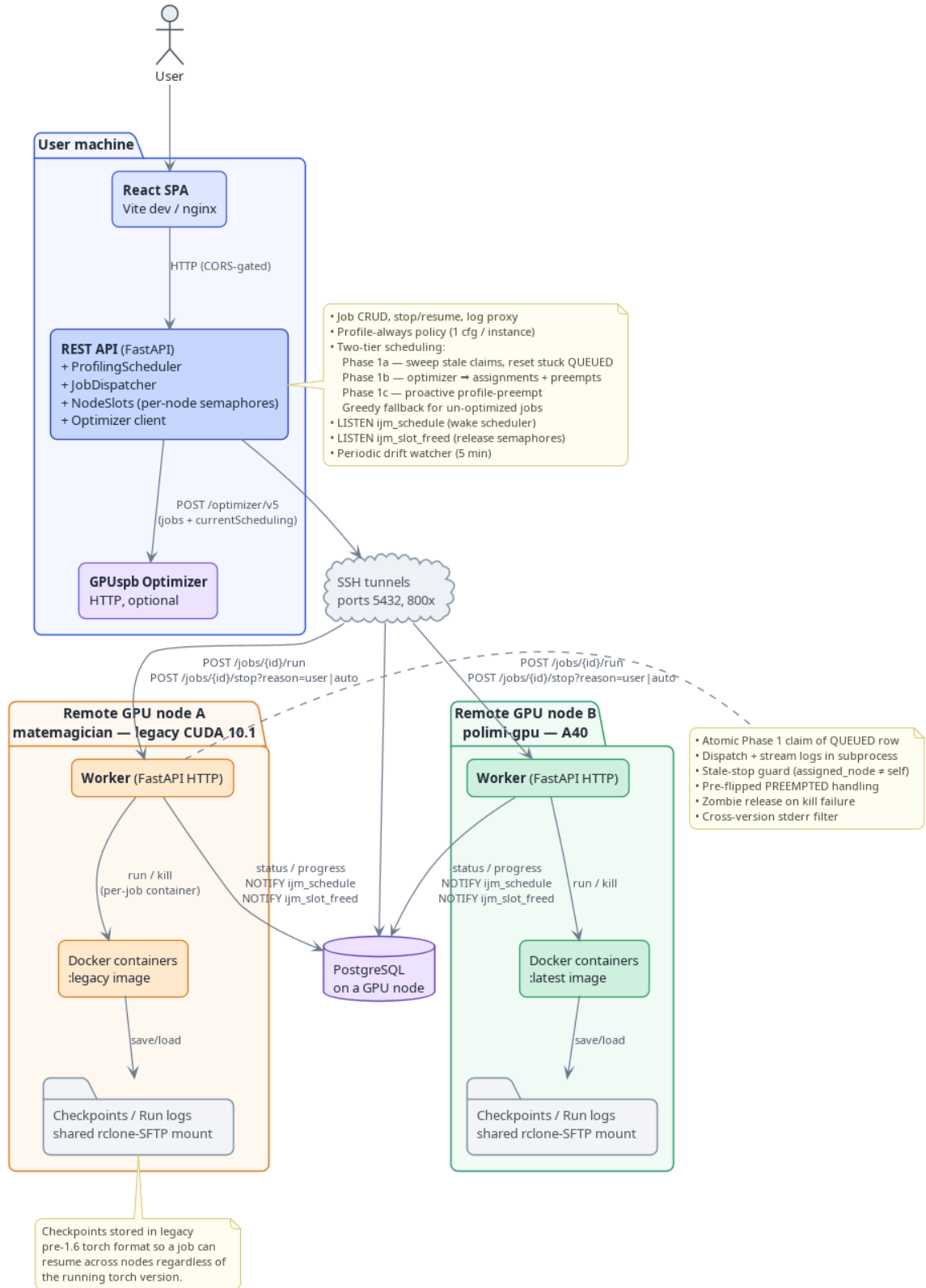
**Table 2.1:** IJM components and their responsibilities.

### 2.1 Deployment topology

IJM runs as a small distributed system. The FastAPI *API* — which also hosts the `ProfilingScheduler`, the `JobDispatcher`, the `NodeSlots` semaphores, and the optimiser client — runs on the operator’s machine. A lightweight FastAPI *worker* runs on every GPU node and owns the local Docker daemon. PostgreSQL runs on one of the GPU nodes.

Communication is plain HTTP over SSH tunnels. Every worker is reached from the API at a stable `http://localhost:800x` alias regardless of its physical location, so the same scheduler code drives a laptop talking to a remote cluster and a single-host development setup. An optional GPUspb *optimiser* HTTP service is reached the same way.

## Current Prototype Architecture (distributed)



**Figure 2.1:** Distributed prototype. The SPA, API, and optimiser run on the operator’s machine; workers and Docker run on each GPU node; PostgreSQL runs on a GPU node. A shared rclone-over-SFTP mount makes checkpoints written by one node visible from the other.

Single-process `docker compose` on the operator’s machine is supported for development but is not the deployment target: any tradeoff between local-dev simplicity and distributed correctness resolves toward the distributed topology.

## 2.2 Scheduling pipeline

The pipeline is shaped by a single governing assumption: *profiling is always worth its cost*. A deep-learning job runs for hours, whereas profiling one GPU configuration takes minutes; leaving a configuration unmeasured risks overlooking the placement that would have saved hours of computation, or missing a hard deadline. The scheduler therefore treats profiling as privileged work that may preempt running jobs, rather than as best-effort measurement deferred until the cluster falls idle. How aggressively the system profiles is a tunable policy: `PROFILING_CONFIGS_PER_JOB` (default 2) sets how many configurations each instance must measure before it is handed to the cost-aware optimiser for standard placement.

A single scheduling pass — `_schedule_waiting_jobs` in `app.py` — handles every dispatch decision. It is invoked whenever a job is submitted, a profiling run finishes, or a slot is freed. A 60-minute watchdog (`_queue_watcher`) serves as a safety net for missed notifications, not as the main wake source. Wake-ups are coalesced through a single `asyncio.Event` and rate-limited to at most one pass every five seconds, so the cluster has time to apply the previous plan before the next one is computed.

The pass runs four phases, all guarded by a single `schedule_lock` except the optimiser’s HTTP call (phase 2), which is made outside the lock so its timeout cannot stall concurrent scheduler work:

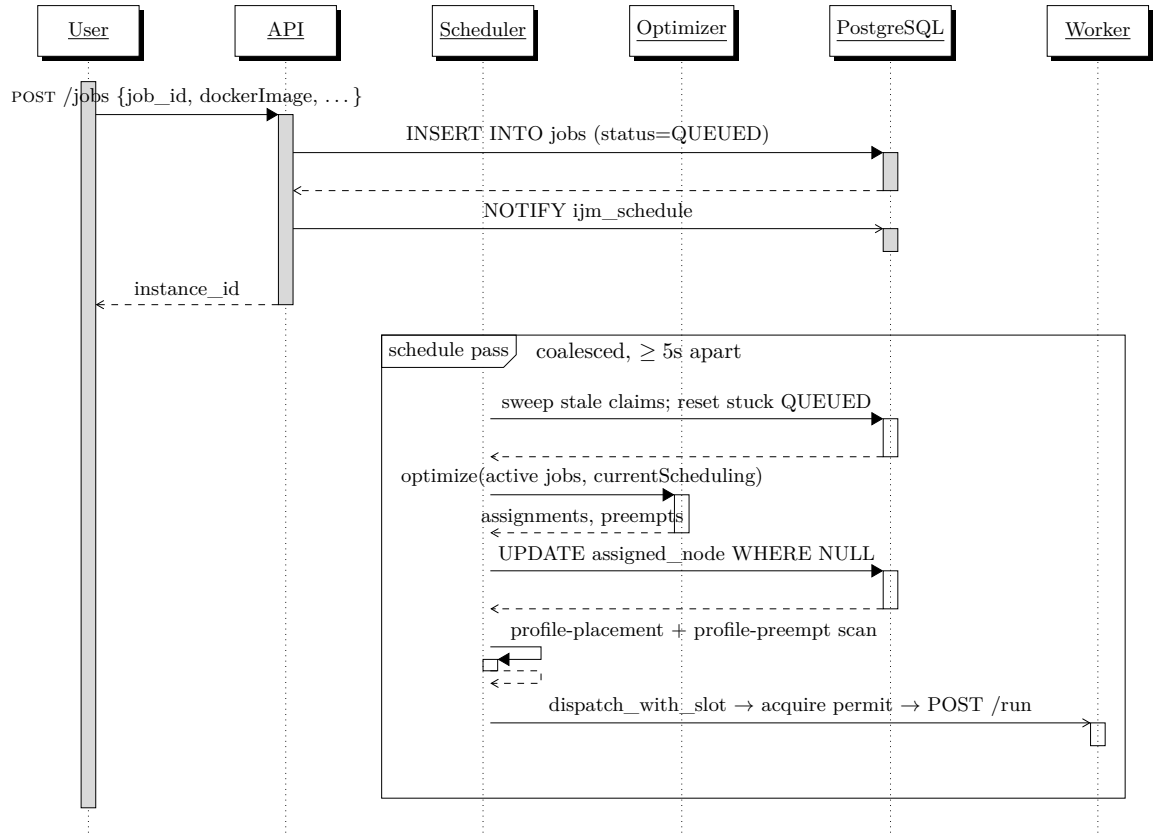
1. **Reset stuck assignments.** Reset `QUEUED` rows whose `assigned_node` was set  $\geq 30$  s ago without the worker advancing them, so a later phase can re-place the row.
2. **Optimiser pass.** Call the optimiser outside the lock (to accommodate its 30 s HTTP timeout) and apply its assignments through a conditional `UPDATE ...WHERE assigned_node IS NULL`, collecting the optimiser’s preempt list. The optimiser only ever scores rows with `is_profiling_run = FALSE` — rows still owing a profile run are invisible to it.
3. **Profile placement.** For `QUEUED` rows that still owe a profile run, the `ProfilingScheduler` atomically reserves the instance’s outstanding unmeasured (`node`, `GPU-count`) cells — up to its `PROFILING_CONFIGS_PER_JOB` budget (default 2) — and dispatches one of them this pass, choosing the node by best fit on remaining capacity; the rest follow on later passes as slots free. This is the mechanism behind the *profile-always* policy: an instance must complete all its profile runs before it becomes a candidate for the optimiser-driven standard placement above.
4. **Profile-preempt.** If a profile-owing row still has no slot after the placement pass, find the cheapest set of low-priority `RUNNING` jobs whose eviction would free enough GPUs for the next unmeasured configuration.

Once the lock is released, the plan is executed asynchronously: each preempt and each dispatch runs as an independent task, so a slow stop on one node never delays placements on another. Before launching a job, the dispatcher must acquire a permit from the destination node’s `NodeSlots` semaphore, whose count is that node’s GPU capacity; the permit is what bounds the number of concurrent jobs per node and arbitrates between tasks competing for the last free slot.

Permits are released asynchronously, because a job’s GPUs are not free the instant the API decides to stop it — only once the worker has killed the container, drained its logs, and writ-

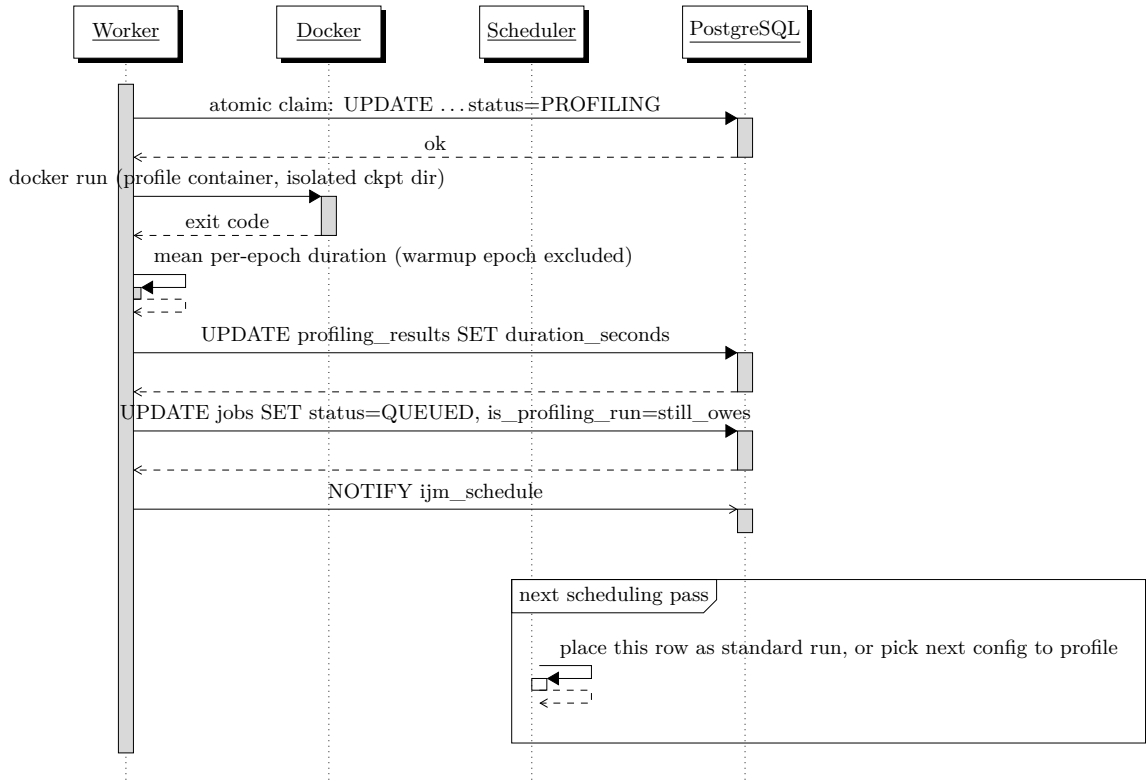
ten a checkpoint. The worker therefore signals each release, and the API returns the permit at that moment, keeping its in-memory count in step with actual occupancy. Each release also records *why* it happened, because the cause determines whether the scheduler should re-plan. A release triggered by an event outside the current plan — a job finishing on its own, an operator stopping a job, or the cleanup of an orphaned container — frees capacity the optimiser did not foresee, and so wakes a fresh scheduling pass. A release caused by the scheduler’s *own* in-flight preempt is the one exception: the plan that issued the preempt already contains the dispatch that will reclaim the freed slot, so re-running the optimiser at that point would let it rewrite a plan it is still executing. Only that case suppresses the re-plan, and doing so is safe because independent watchdogs — a drift heartbeat, the stuck-dispatch reaper, and a periodic queue watcher — re-wake the scheduler if a planned dispatch ever fails to land.

Figure 2.2 shows the submission path, Figure 2.3 the cross-instance profile cycle, Figure 2.4 an optimiser-driven preempt and migrate round, and Figure 2.5 the standard run.

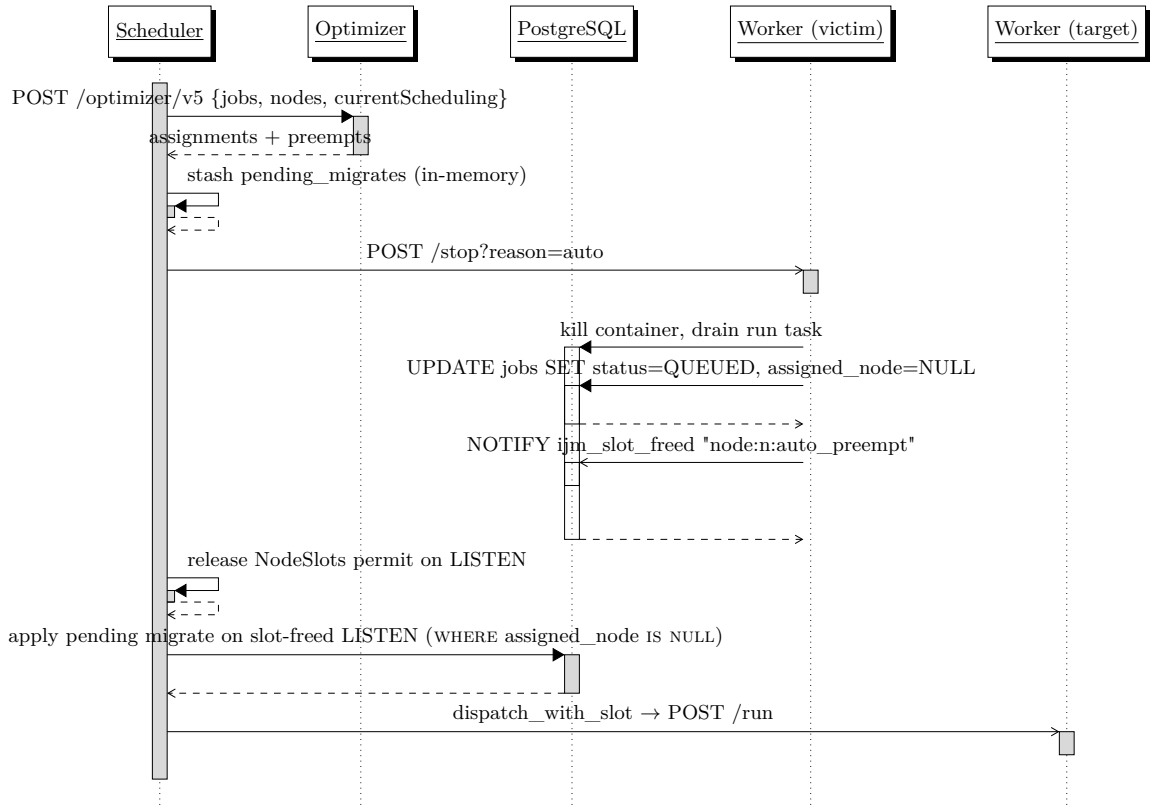


**Figure 2.2:** Submission and scheduling. The API writes the row and notifies the scheduler; placement is decided by the next pass. The optimiser call sits outside the schedule lock so its 30 s HTTP timeout cannot block other passes.

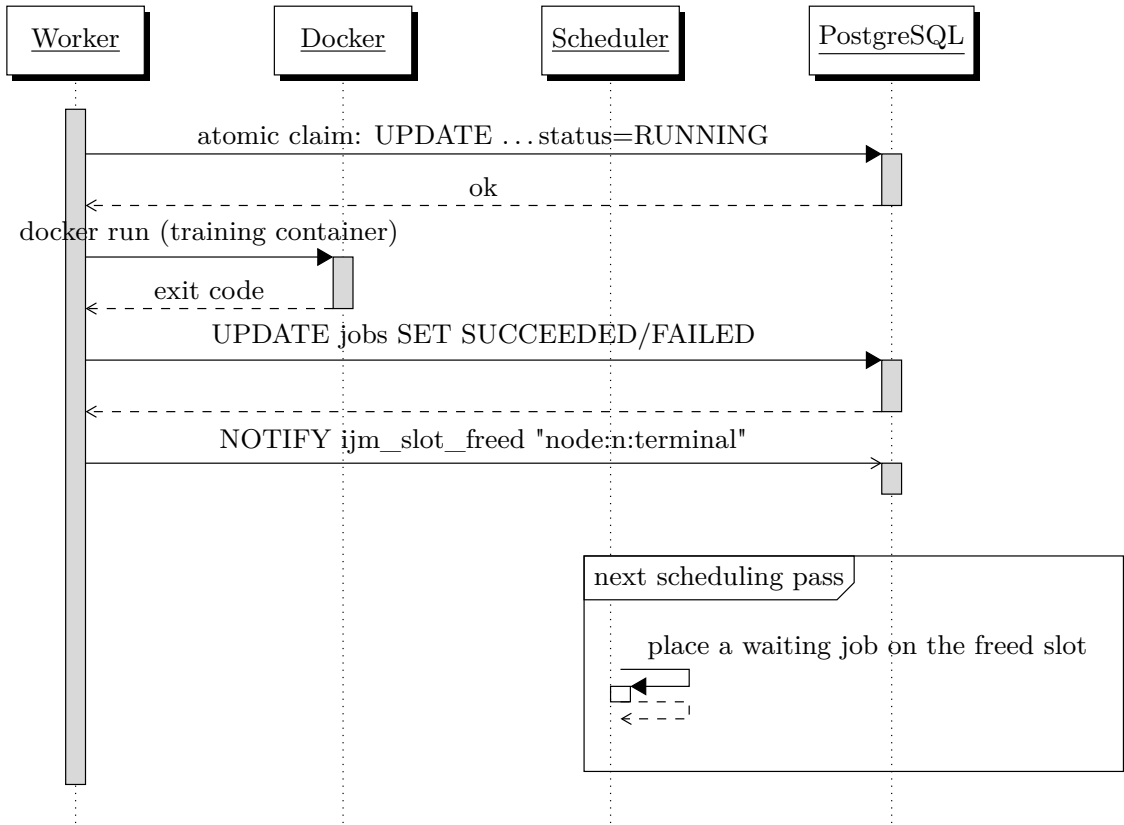




**Figure 2.3:** Profiling cycle. Profile measurements are keyed by the (job-type, GPU-config) pair and shared across submissions of the same type. Each instance profiles up to `PROFILING_CONFIGS_PER_JOB` (default 2) configurations before it becomes a standard-run candidate; the per-instance claim row guards against duplicate measurements. The worker records the result and re-queues the row in a single transaction (`worker/profiling.py:handle_complete`).



**Figure 2.4:** Optimiser-driven preempt and migrate. The API trusts the GPUsp plan: any job in `currentScheduling` that the optimiser drops or moves is preempted with `reason=auto`, clearing its assignment. Because an `auto_preempt` release deliberately does *not* wake a fresh optimiser pass, a moved job’s new placement is stashed in `state.pending_migrates` and re-applied by the slot listener (conditional `UPDATE ...WHERE assigned_node IS NULL`) once the source `/stop` has drained the row to `QUEUED`.



**Figure 2.5:** Standard run. The worker performs an atomic conditional `UPDATE ...WHERE status = ANY(RUNNABLE_STATUSES)` so concurrent stops or duplicate dispatches cannot race a row into `RUNNING`. On exit the worker fires a single `ijm_slot_freed` (reason `terminal`) in the same transaction as the status flip; the slot listener releases the permit and wakes the scheduler, so the terminal path needs no separate `ijm_schedule`.

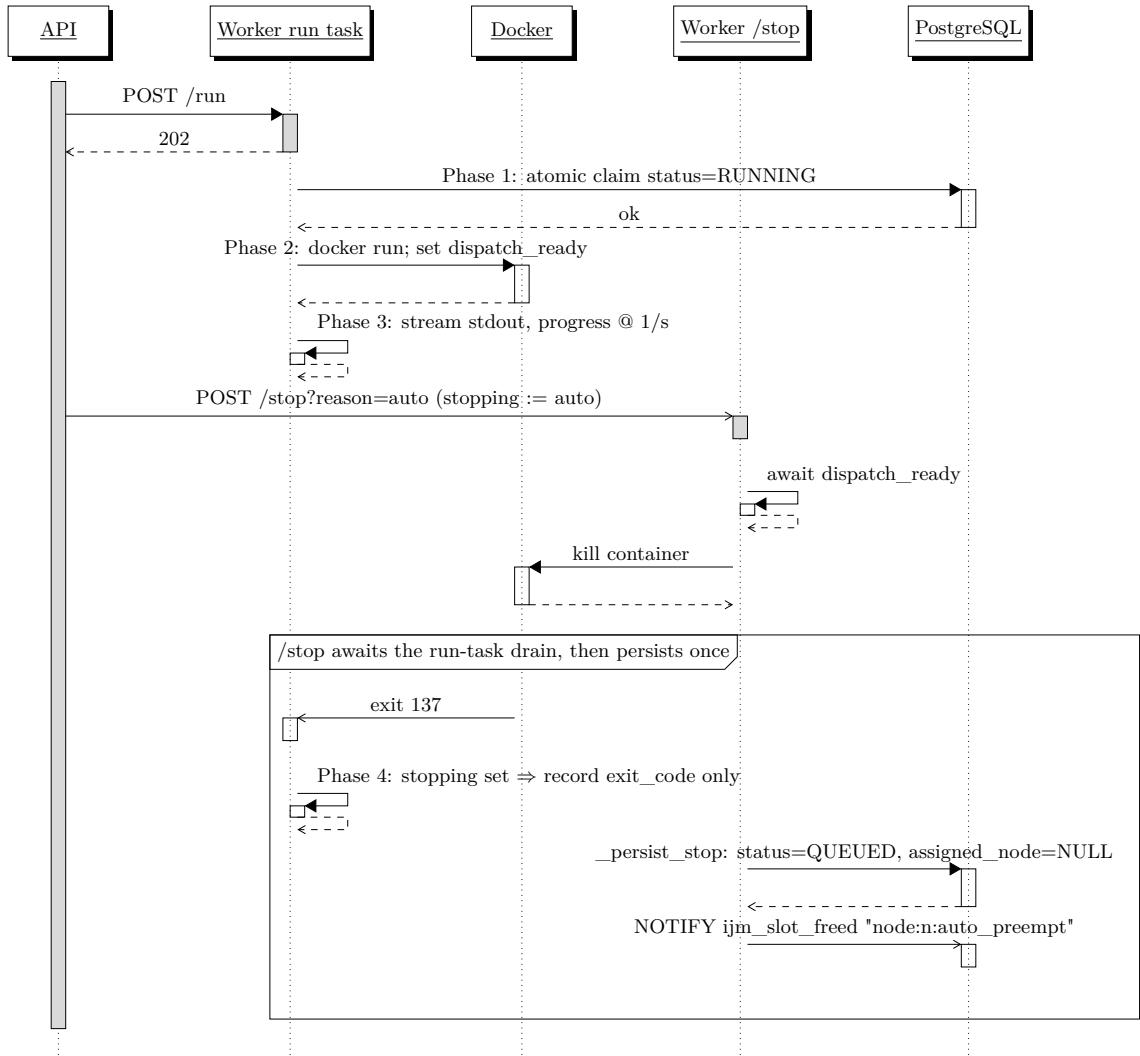
## 2.3 Worker execution lifecycle

The scheduling pass decides *where* a job runs; the worker's run task (`_run_job`) owns *how* it runs and how it stops. The task moves through four phases (Table 2.2), and every database transition is a conditional `UPDATE`, so a racing `/stop`, the stuck-dispatch reaper, or a duplicate dispatch cannot corrupt the row.

| Phase       | Action                                                                                                                                   | Invariant it enforces                                                                                                                                                                                                                         |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Claim    | Atomic UPDATE<br>... <b>status</b> =RUNNING/PROFILING WHERE<br><b>status</b> ∈ RUNNABLE AND<br><b>assigned_node</b> =self                | A <b>/stop</b> or reaper that already moved the row makes the claim miss, so no container starts for a re-placed row.                                                                                                                         |
| 2. Launch   | Pick distinct physical GPU indices, prepare the per-job checkpoint/runs directories, <b>docker run</b>                                   | Distinct indices stop two 1-GPU jobs both landing on GPU 0; an in-process placeholder is set <i>before</i> any <b>await</b> , so a <b>/stop</b> cannot slip into the window before the container exists.                                      |
| 3. Stream   | Parse <b>Epoch x/y</b> from container stdout; flush the latest value through a once-per-second background writer                         | The progress UPDATE is conditional on a matching <b>status</b> and <b>container_name</b> , so a reassigned row no-ops it and stops the stream. Coalescing replaces a per-epoch round-trip that fell behind over the SSH-tunnelled connection. |
| 4. Complete | Write the terminal status and fire one <b>ijm_slot_freed</b> ; or, if the row was migrated or already stopped, record only the exit code | A kill-induced exit 137 never clobbers the new owner’s RUNNING; a failed run also drops its outstanding profile claims so their cells are not left reserved.                                                                                  |

**Table 2.2:** The worker run task’s four phases — distinct from the scheduling pass’s four phases above: that pass plans, this task executes.

A **/stop** and its target run task are separate coroutines that must not race. The handler tags the row in an in-process **stopping** map, waits for the run task to publish its container (**dispatch\_ready**), kills it, and only *after* awaiting the run task’s full drain writes the final status and frees the slot (Figure 2.6). The run task’s own completion phase, seeing the **stopping** tag, records just the exit code, so the slot is freed exactly once. The same hazard across nodes is closed on the API side: **dispatch\_with\_slot** awaits any in-flight **/stop** on the target node before posting **/run**, so a migration’s new container never starts before the evicted one’s CUDA context has released its GPU.



**Figure 2.6:** Worker execution and a scheduler stop. The run task and the concurrent `/stop` synchronise through the in-process `dispatch_ready` and `stopping` maps plus a drain await, so the slot is freed exactly once and by exactly one path. A `reason=auto` stop returns the row to `QUEUED` with its assignment cleared (a `reason=user` stop lands on `PREEMPTED` instead); the resulting `auto_preempt` notify is the one the slot listener swallows (§2.2).

## 2.4 State machine

Table 2.3 lists every transition a job row can take. All transitions are driven by atomic conditional `UPDATEs`, so concurrent stop, dispatch, and completion paths cannot clobber each other's work.

| From               | To                  | Trigger                                                                                             |
|--------------------|---------------------|-----------------------------------------------------------------------------------------------------|
| QUEUED             | PROFILING / RUNNING | Worker Phase 1 atomic claim                                                                         |
| PROFILING          | QUEUED              | Profile run finished; <code>is_profiling_run</code> flips to <code>false</code>                     |
| PROFILING          | FAILED              | Profile container exited non-zero                                                                   |
| RUNNING            | SUCCEEDED / FAILED  | Container exit                                                                                      |
| RUNNING            | PREEMPTED           | User <code>POST /jobs/{id}/stop?reason=user</code>                                                  |
| RUNNING            | QUEUED              | Scheduler preempt ( <code>reason=auto</code> ): clears <code>assigned_node</code> for the next pass |
| PREEMPTED / FAILED | QUEUED              | <code>POST /jobs/{id}/resume</code>                                                                 |

**Table 2.3:** Job state transitions.

Two distinct stop reasons separate *user-initiated* preemption from *scheduler-initiated* preemption. A user stop is sticky: PREEMPTED stays put until the user explicitly resumes. A scheduler stop returns the row to QUEUED with a cleared assignment, so the next pass can place it elsewhere automatically. The worker enforces this distinction and ignores stale `/stop` calls aimed at a row that has since been re-assigned to another node.

## 2.5 Persistent state

Two PostgreSQL tables hold every fact the scheduler reads or writes; both are created idempotently on startup from `backend/schema.sql`.

```
CREATE TABLE jobs (
    id TEXT PRIMARY KEY,
    job_id TEXT NOT NULL,
    image TEXT NOT NULL,
    command JSONB NOT NULL,
    script_path TEXT,
    directory_to_mount TEXT,
    status TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL,
    container_name TEXT,
    exit_code INT,
    progress TEXT,
    priority INT DEFAULT 3,
    deadline TIMESTAMPTZ,
    batch_size INT,
    epochs_total INT,
    profiling_epochs_no INT,
    assigned_node TEXT,
    assigned_gpu_config JSONB,
    is_profiling_run BOOLEAN DEFAULT FALSE
);

CREATE TABLE profiling_results (
    id TEXT PRIMARY KEY,
    job_id TEXT NOT NULL,
    instance_id TEXT,
    gpu_config JSONB NOT NULL,
    node_id TEXT NOT NULL,
    duration_seconds FLOAT,
    created_at TIMESTAMPTZ NOT NULL
);
```

```
);
```

Two columns of `jobs` are load-bearing for the scheduling pass above: `assigned_node` is the target the optimiser pass writes under `WHERE assigned_node IS NULL`, and `is_profiling_run` is the flag that makes a row invisible to the optimiser (phase 2) and visible to the `ProfilingScheduler` (phases 3–4).

A unique index on (`job_id`, `gpu_config`) makes the cross-instance profile claim atomic. Profile measurements are keyed by the *job type* (`job_id`) and the GPU configuration, so two concurrent submissions of the same type race to own each unmeasured cell; the loser’s `INSERT` trips the unique constraint and the winner’s measurement is shared by every subsequent instance of that type.

## 2.6 Concurrency invariants

The control plane relies on three invariants, each defended in code.

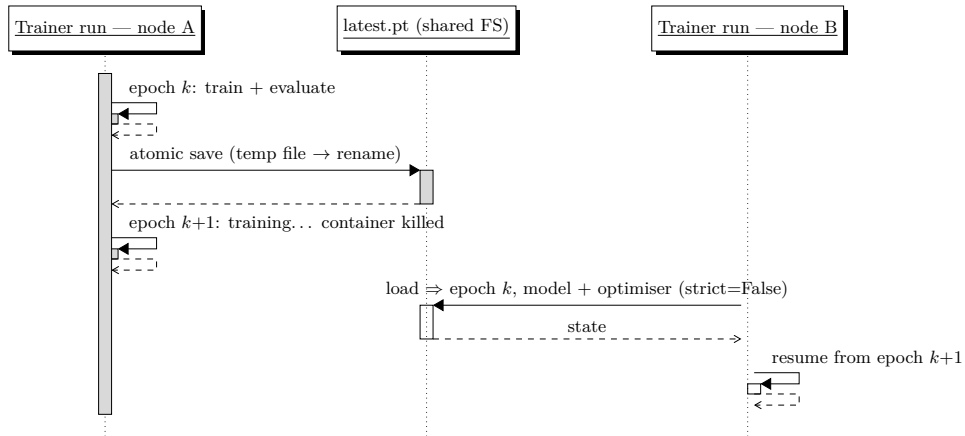
**No GPU oversubscription.** Every dispatch goes through `NodeSlots.acquire(node_id, n_gpus)`. On startup the semaphores are reconciled from the database: only rows in `RUNNING` or `PROFILING` pre-acquire their permits, since a `QUEUED` row with an assigned node has no live container yet (its dispatch task acquires the permit just before posting `/run`). A periodic drift watcher recomputes the authoritative count at the `IJM_DRIFT_HEARTBEAT_S` cadence (default 15 s) and corrects any divergence.

**No duplicate containers per row.** Both the worker (Phase 1 claim, `running_jobs[job_id]` check) and the API (`dispatch_tasks[instance_id]` deduplication) refuse a second concurrent dispatch for the same instance. The reaper that resets stuck `QUEUED` rows cancels any live dispatch task instead of double-releasing its permit.

**Profiling is sacred.** Auto-preempt (`_preempt_and_release`) refuses to act on `is_profiling_run=TRUE`, and profile-preempt (phase 4) filters victims by `status='RUNNING'` only. Only an explicit user stop can cancel an in-flight measurement.

## 2.7 Stoppable, resumable, cross-node training

A job is *stoppable* because the trainer persists its state after every epoch, and *resumable* because every (re)start loads that state. `BaseTrainer.train` writes `latest.pt` at the end of each epoch through a temp-file-and-rename, so the on-disk checkpoint is always a whole epoch, never a half-written file. Stopping a job is therefore just killing its container (§2.3): the in-progress epoch is discarded, every completed epoch is durable. The next dispatch — on the same node or another — reconstructs the trainer, which loads `latest.pt`, restores the model, optimiser, and `current_epoch`, and continues the loop (Figure 2.7). No scheduler state mirrors training progress; the checkpoint file is the single source of truth for how far a job has run.



**Figure 2.7:** Stop and resume. The trainer checkpoints after each epoch with a temp-file-and-rename, so `latest.pt` is never half-written. A stop kills the container mid-epoch; the next run — here on a different node, possibly a different torch version — loads `latest.pt` and continues. Only the interrupted epoch  $k+1$  is recomputed.

Two GPU generations coexist, so a resume must survive a change of torch version mid-job. The `matemagician` node is pinned to driver 418.x (CUDA 10.1 max), so it runs a separate `:legacy` image built on PyTorch 1.5.1 and Python 3.7; the remaining nodes run the default `:latest` image on PyTorch 2.6 and CUDA 12.4. Both images write the checkpoint in the legacy pre-1.6 `torch.save` format, and the load is deliberately lenient: `strict=False` tolerates state-dict key differences between versions, and a failed optimiser-state load restarts Adam from scratch rather than aborting the resume — the model weights are what matter, and a few epochs of optimiser warmup are invisible at scenario timescales. This is what lets `cnn_big` migrate between a modern A40 and a legacy P600 without losing progress.

The shared `data/` directory is mounted on every node via `rclone` over SFTP. It contains:

- `data/checkpoints/{job_id}/` — per-job checkpoints, mounted into the container at `/checkpoints`;
- `data/runs/{job_id}/` — captured stdout (`output.log`) and TensorBoard artifacts, mounted at `/runs`;
- `data/shared/data/` — read-only dataset cache (MNIST, CIFAR-10), mounted at `/runs/data`.

Two worker-side environment variables let the same `nodes_config.json` drive heterogeneous nodes. `WORKER_GPU_MODE` selects how Docker exposes GPUs to the training container: `runtime` for the standard NVIDIA runtime, `cdi` for the rootless daemon on `polimi-gpu`, and `none` for a CPU-only sandbox. `IMAGE_TAG_OVERRIDE` rewrites the image tag, which is how `matemagician` swaps `:latest` for `:legacy`.



## 3. Configuration and Job Submission

IJM is configured at three layers: a pair of JSON files that describe the cluster, an HTTP submission API for jobs, and a handful of runtime environment variables. This section documents each in turn.

### 3.1 Cluster configuration

The cluster topology is loaded at startup from two JSON files, whose paths are configurable via environment variables.

#### 3.1.1 Nodes (`nodes_config.json`)

Each entry describes a compute node and its GPU resources. The example below is the deployment used throughout Chapter 4 (`config/nodes_config.tunnel.json`); the file checked in at `config/nodes_config.json` is a mock cluster used by the unit tests.

```
[
  {
    "id": "matemagician",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8001",
    "cost": 0.10,
    "resources": [
      { "gpu_type": "QuadroP600", "gpu_count": 2 }
    ]
  },
  {
    "id": "polimi-gpu",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8002",
    "cost": 0.30,
    "resources": [
      { "gpu_type": "A40", "gpu_count": 2 }
    ]
  }
]
```

| Field                 | Type   | Description                                                                                                                                                                                               |
|-----------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id                    | string | Unique node identifier                                                                                                                                                                                    |
| isForProfiling        | bool   | If <b>true</b> , the node can host profiling runs. Production jobs may still land on it; profiling jobs may only land on it                                                                               |
| workerUrl             | string | HTTP base URL of the node's worker, reached through an SSH tunnel. The API dispatches by posting <code>/jobs/{id}/run</code> and <code>/jobs/{id}/stop</code> here; every node must be backed by a worker |
| cost                  | float  | Idle cost of the node (\$/h), used by the optimiser                                                                                                                                                       |
| resources             | array  | GPU resources attached to the node                                                                                                                                                                        |
| resources[].gpu_type  | string | GPU model name (e.g. A40, L40S, QuadroP600)                                                                                                                                                               |
| resources[].gpu_count | int    | Number of GPUs of this type                                                                                                                                                                               |

**Table 3.1:** Node configuration fields.

GPU type and count are declared explicitly rather than auto-detected. This admits mixed-GPU nodes that ANDREAS did not support (e.g. a node with both A40 and L40S GPUs), and it lets the operator declare nodes that are not physically attached to the API host — the API only ever talks to workers over HTTP.

### 3.1.2 Energy costs (gpu\_energy\_costs.json)

A lookup table that maps a GPU type and bundle size to an hourly energy cost (\$/h). The scheduler consults this table when scoring candidate placements.

```
{
  "A40": { "1": 0.30, "2": 0.55, "4": 1.05 },
  "L40S": { "1": 0.40, "2": 0.75, "4": 1.40 },
  "QuadroP600": { "1": 0.03, "2": 0.055 },
  "Blackwell": { "1": 1.00, "2": 1.85, "4": 3.50 }
}
```

## 3.2 Job submission

**Endpoint:** POST `/jobs`

### 3.2.1 Request body

```
{
  "job_id": "LSTM-small",
  "dockerImage": "ijm-lstm-small:dev",
  "scriptPath": "train.py",
  "directoryToMount": "/data/my-project",
  "Priority": 3,
  "deadline": "2026-03-20T18:00:00",
  "batchSize": 64,
  "profilingEpochsNo": 3,
  "epochsTotal": 20
}
```

| Field             | Type         | Required | Description                                                                                                                                                                               |
|-------------------|--------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| job_id            | string       | yes      | Reusable job type identifier; profile measurements are correlated across submissions sharing the same <code>job_id</code>                                                                 |
| dockerImage       | string       | yes      | Docker image to execute                                                                                                                                                                   |
| scriptPath        | string       | no       | Path to the training script (overrides Docker CMD with <code>python3 &lt;scriptPath&gt;</code> )                                                                                          |
| directoryToMount  | string       | no       | Host directory persisted on the job row (ANDREAS-compatibility field); not mounted by the current worker, which mounts only checkpoints, the runs directory, and the shared dataset cache |
| command           | string[]     | no       | Command and arguments (defaults to the image entrypoint; overridden by <code>scriptPath</code> )                                                                                          |
| Priority          | int 1–5      | no       | Job priority (default 3)                                                                                                                                                                  |
| deadline          | datetime     | no       | Job deadline (ISO 8601)                                                                                                                                                                   |
| batchSize         | int          | no       | Batch size passed as the <code>BATCH_SIZE</code> env var                                                                                                                                  |
| profilingEpochsNo | int $\geq 1$ | no       | Epochs per profiling run (default 3)                                                                                                                                                      |
| epochsTotal       | int $\geq 1$ | no       | Total training epochs (default 20)                                                                                                                                                        |

**Table 3.2:** Job creation request fields.

Each submission receives a unique UUID instance ID (`id`) for lifecycle tracking, separate from the `job_id` type identifier. When `scriptPath` is provided the container runs `python3 <scriptPath>`; otherwise the image’s default CMD is used. Unlike ANDREAS, `Priority` and `deadline` are optional, and an additional `command` field gives flexibility for non-standard workloads.

### 3.2.2 Response (201 Created)

The response carries the full job object, including the scheduler’s initial decision: assigned node, GPU configuration, and whether the first run will be a profile or a standard execution.

```
{
  "id": "a1b2c3d4-...",
  "job_id": "LSTM-small",
  "image": "ijm-lstm-small:dev",
  "command": ["python3", "train.py"],
  "status": "QUEUED",
  "is_profiling_run": true,
  "assigned_node": "polimi-gpu",
  "assigned_gpu_config": { "A40": 1 },
  "priority": 3,
  "epochs_total": 20,
  "profiling_epochs_no": 3,
  "created_at": "2026-03-18T10:00:00Z",
  "updated_at": "2026-03-18T10:00:00Z",
  ...
}
```

### 3.3 Full API overview

Table 3.3 lists every endpoint. Interactive documentation is available at `/docs` (Swagger UI) and `/redoc` when the API is running.

| Method | Path                                 | Description                                                                       |
|--------|--------------------------------------|-----------------------------------------------------------------------------------|
| GET    | <code>/</code>                       | Health check endpoint                                                             |
| GET    | <code>/health</code>                 | Detailed health check — pings DB and checks job runner                            |
| GET    | <code>/nodes</code>                  | List all cluster nodes with their current status                                  |
| GET    | <code>/gpu-costs</code>              | Hourly GPU energy costs used by the scheduler for optimisation                    |
| GET    | <code>/configurations</code>         | List all valid hardware configurations in the cluster                             |
| POST   | <code>/jobs</code>                   | Create a new job                                                                  |
| GET    | <code>/jobs</code>                   | List all jobs ordered by creation time (paginated via <code>limit/offset</code> ) |
| GET    | <code>/jobs/{id}</code>              | Get a specific job by ID                                                          |
| POST   | <code>/jobs/{id}/stop</code>         | Request job to be stopped (user-driven; sticky PRE-EMPTED state)                  |
| POST   | <code>/jobs/{id}/resume</code>       | Request job to be resumed                                                         |
| DELETE | <code>/jobs/{id}</code>              | Delete a job, killing any running container first                                 |
| DELETE | <code>/jobs</code>                   | Delete all jobs and their profiling results                                       |
| GET    | <code>/jobs/{id}/logs</code>         | Return the output log for a job by proxying to the worker that owns it            |
| GET    | <code>/profiling-results/{id}</code> | All profiling results for a job, ordered by duration (fastest first)              |
| GET    | <code>/admin/slots</code>            | NodeSlots snapshot: in-memory <code>used</code> vs. DB-authoritative count        |
| POST   | <code>/admin/reconcile-slots</code>  | Force NodeSlots to match DB-authoritative state (idempotent)                      |
| GET    | <code>/admin/dispatch-tasks</code>   | List in-flight <code>_dispatch_when_slot_free</code> tasks                        |

**Table 3.3:** API endpoints.

Three architectural choices set the API apart from ANDREAS. First, scheduling, profiling, and dispatch are consolidated into a single FastAPI service, in contrast to ANDREAS’s split between a Java Jobs Manager and a C++ Jobs Optimiser. Second, execution lives in per-node *workers* — also FastAPI — that the API reaches over HTTP through SSH tunnels; the external GPUspb optimiser is reached the same way and owns every standard-run placement decision. Third, IJM exposes explicit `stop`, `resume`, `logs`, and `delete` endpoints, together with `/profiling-results` and `/configurations`, so the user has direct visibility into the scheduler’s state.

### 3.4 Runtime environment

Table 3.4 lists the runtime knobs that influence the scheduler. Unless noted otherwise, every variable is read by the API process at startup.

| Variable                     | Default              | Description                                                                                                                                                |
|------------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DATABASE_URL                 | postgresql://.../ijm | PostgreSQL DSN, reached via SSH tunnel                                                                                                                     |
| OPTIMIZER_URL                | unset                | The GPUspb HTTP service used for standard-run placement                                                                                                    |
| OPTIMIZER_METHOD             | RG                   | GPUspb solver method passed in the request body                                                                                                            |
| OPTIMIZER_VERBOSE            | 0                    | Per-call log verbosity (0–3)                                                                                                                               |
| PROFILING_CONFIGS_PER_JOB    | 2                    | How many distinct GPU configurations a single instance profiles before falling through to a standard run                                                   |
| IJM_DRIFT_HEARTBEAT_S        | 15                   | Period of the slot-tracker drift heartbeat that reconciles <code>mem_used</code> against <code>db_used</code> per node                                     |
| WORKER_GPU_MODE              | runtime              | Worker-side; how Docker exposes GPUs to training containers ( <code>runtime</code> / <code>cdi</code> / <code>none</code> )                                |
| IMAGE_TAG_OVERRIDE           | unset                | Worker-side; rewrites <code>:latest</code> to the given tag. Used on <code>matemagician</code> to pick the CUDA-10.1-compatible <code>:legacy</code> image |
| HOST_ROOT, HOST_PROJECT_ROOT | /host                | Resolve container-internal vs. host paths for Docker bind mounts; <code>HOST_PROJECT_ROOT</code> falls back to <code>HOST_ROOT</code> when unset           |

**Table 3.4:** Key environment variables.

## 4. End-to-End Scenarios

This section presents the cost model and two end-to-end scenarios on the two-node cluster, both probing how GPUsp’s `method=RG` proxy behaves. Scenario 1 (*under-subscribed*, single type) isolates its *horizon-myopia*: an urgent job is parked on a slow but free bundle because its deadline overrun is invisible at the next-event horizon. Scenario 2 (*full*: A40 held by `cnn_big` pins, P600 by slack `lstm-small`) shows the same myopia under capacity pressure — no preemptible bundle meets the urgent deadline, so the proxy drops the cheapest slack, parks the job on the cheapest slow bundle, and the deadline is recovered only by a later migration onto freed A40. The priority-aware outcome is emergent (§4.1): the optimiser minimises a stateless plan cost, never reasoning about preemption or migration.

### 4.1 Objective and cost model

IJM delegates standard-run placement to GPUsp run with `method=RG`. We report what that optimiser *actually* minimises, since it is the ground truth for every placement we observe: per scheduling decision, Random Greedy scores a candidate plan with the proxy `proxy_function_min` (`optimizer.cpp`):

$$\begin{aligned} \text{COST}_{\text{RG}} = & \sum_{\text{sched } j} \left( r(b_j) \Delta t + w(p_j) [t_0 + \Delta t - d_j]^+ \right) + \sum_n \text{cost}_n \Delta t \\ & + \lambda \sum_{\text{unsched } j} w(p_j) [t_0 + T_j^{\text{full}} - d_j]^+, \end{aligned}$$

with indices  $j$  (over the plan’s scheduled/unscheduled jobs) and  $n$  (over the cluster’s nodes), and parameters ( $[x]^+ \equiv \max(0, x)$ ):

- $\Delta t$  — next-event horizon: the soonest finish among the plan’s scheduled jobs;
- $t_0$  — the current time;
- $b_j, p_j, d_j$  — job  $j$ ’s assigned GPU bundle, priority, and deadline;
- $r(b)$  — hourly rate of GPU bundle  $b$  (Table 4.2, left);
- $w(p)$  — per-priority tardiness weight (Table 4.2, centre);
- $\text{cost}_n$  — per-node operating cost from `nodes_config.json` (Table 4.2, right), passed through in the optimiser request. Note the sum is over *every* configured node: `compute_nodeCost` charges  $\text{cost}_n \Delta t$  whether or not the node hosts a job (no occupancy filter), so this term gives no consolidation incentive;
- $T_j^{\text{full}}$  — job  $j$ ’s full remaining run time on its *slowest profiled bundle* (the max over the job’s measured configs, `get_maxExecTime`), charged in full and *not* clipped to  $\Delta t$ ;
- $\lambda$  — unscheduled-job penalty multiplier, hard-coded to  $\lambda=100$  in `proxy_function_min`.

Three features of this objective (Table 4.1) drive everything the scenarios show.

| Proxy feature                               | Charged                                              | Consequence                                                                                                         |
|---------------------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| GPU/tardiness/node of <i>scheduled</i> jobs | only to horizon $\Delta t$                           | a placement that over-runs a far-off deadline still reads zero tardiness <i>now</i> (horizon-myopia; Scenario 1)    |
| <i>Unscheduled</i> job                      | $\lambda=100\times$ <i>full</i> worst-case tardiness | omitting a tight high-priority job is ruinous, a slack low-priority one nearly free (drop-the-cheapest; Scenario 2) |
| Migration / preemption                      | no term                                              | the plan is scored alone, ignoring what runs — in-proxy, eviction is free                                           |

**Table 4.1:** The three features of  $\text{COST}_{\text{RG}}$  that drive both scenarios.

$T_j^{\text{full}}$  is the full run on a job’s *slowest profiled* bundle, so it jumps as the sweep measures slower hardware (QuadroP600 is  $\sim 15\times$  slower than A40 for `cnv_big`, Table 4.4): a long tight-deadline job can flip from free-to-omit to *must-schedule* with no change in free capacity, which Scenario 1’s sweep (§4.3) exercises.

| GPU bundle            | \$/hour | Priority | TardWeight (\$/sec) | Node         | $\text{cost}_n$ (\$/h) |
|-----------------------|---------|----------|---------------------|--------------|------------------------|
| QuadroP600 $\times$ 1 | 0.030   | 1        | 0.000254            |              |                        |
| QuadroP600 $\times$ 2 | 0.055   | 2        | 0.000301            |              |                        |
| A40 $\times$ 1        | 0.300   | 3        | 0.000349            | matemagician | 0.10                   |
| A40 $\times$ 2        | 0.550   | 4        | 0.000396            | polimi-gpu   | 0.30                   |
|                       |         | 5        | 0.000444            |              |                        |

**Table 4.2:** Bundle hourly rates (left), per-priority tardiness weights (centre), and per-node operating cost  $\text{cost}_n$  (right).

The cluster used throughout this section is two profiling-capable nodes, four slots total: `matemagician` with  $2\times$ QuadroP600 and `polimi-gpu` with  $2\times$ A40. Hourly rates are as in Table 4.2. For each new job type the profile sweep exercises all four (`node`, `GPU-count`) cells; with `configs_per_job=2` two instances of the type suffice to cover all four.

#### 4.1.1 Judging the proxy: an idealised yardstick

$\text{COST}_{\text{RG}}$  is what the optimiser minimises, but it is a *per-decision* surrogate. To judge its choices we compare against an *idealised* full-run cost — the same terms, but with tardiness (and GPU/node cost) measured at each job’s *real* completion instead of at  $\Delta t$ . The gap between the two is what the scenarios expose:

- **Horizon-myopia (free slot  $\Rightarrow$  Scenario 1).** Charged only to  $\Delta t$ , tardiness reads zero while a deadline is still in the future at the horizon — so the proxy parks an urgent job on a cheap, free, far-too-slow bundle rather than faster hardware it would have to clear first.
- **Drop-the-cheapest (full cluster  $\Rightarrow$  Scenario 2).** When jobs exceed capacity, every plan leaves someone unscheduled, and the  $\lambda=100$  penalty drops whoever is cheapest to omit — smallest  $w(p)$  [lateness]<sup>+</sup>. On the true per-epoch costs the same myopia also

governs *which* preemptible bundle then absorbs the urgent job (the cheapest, not the least-tardy), so Scenario 2 exhibits both effects.

- **Preemption is a side effect.** The objective is stateless (§4.1): the optimiser picks a target plan, and IJM realises it by stopping whatever the plan dropped — the optimiser never reasons about preemption.
- **method=PR would differ.** The deadline-aware variant scores per-job *real* finish times and would dodge the horizon-myopia.

## 4.2 Available job types

| Type                      | Architecture                                                 | Dataset                             |
|---------------------------|--------------------------------------------------------------|-------------------------------------|
| <code>lstm-small</code>   | LSTM 1-layer, 128 hidden                                     | MNIST (sequence)                    |
| <code>lstm-big</code>     | LSTM 3-layer, 256 hidden, dropout 0.2                        | MNIST (sequence)                    |
| <code>convnet</code>      | 3-layer CNN + BN                                             | CIFAR-10                            |
| <code>efficientnet</code> | MBCov EfficientNet                                           | CIFAR-10                            |
| <code>cnn_big</code>      | 10-block CNN, $3 \rightarrow \dots \rightarrow 512$ channels | synthetic $128 \times 128 \times 3$ |

**Table 4.3:** Training containers under `runtime/`. All share the same checkpoint contract (legacy pre-1.6 `torch.save`, loaded with `strict=False`), so any job can resume across nodes regardless of the destination’s torch version.

| Type                    | A40×1 [s/ep] | A40×2 [s/ep] | QuadroP600×1 [s/ep] | QuadroP600×2 [s/ep] |
|-------------------------|--------------|--------------|---------------------|---------------------|
| <code>lstm-small</code> | 5.88         | 8.61         | 9.17                | 17.71               |
| <code>cnn_big</code>    | 5.08         | <b>4.55</b>  | 78.36               | <b>46.66</b>        |

**Table 4.4:** Steady-state per-epoch compute from `profiling_results` (mean of post-warmup epochs, 2026-06-25 runs; the runtime’s own per-epoch timing, after the profiler-measurement fix). Bold marks 2-GPU bundles that beat their 1-GPU counterpart on wall-clock. `cnn_big` wins  $1.12\times$  on A40×2 and  $1.68\times$  on P600×2; `lstm-small` has no 2-GPU win (DataParallel sync dominates). `cnn_big` is the only type used by Scenario 1.

| Type                    | A40×1 [\$/ep] | A40×2 [\$/ep] | P600×1 [\$/ep] | P600×2 [\$/ep] |
|-------------------------|---------------|---------------|----------------|----------------|
| <code>lstm-small</code> | 0.000490      | 0.001315      | 0.0000764      | 0.000271       |
| <code>cnn_big</code>    | 0.000423      | 0.000695      | 0.000653       | 0.000713       |

**Table 4.5:** GPUcost per epoch, computed as  $\text{per-epoch[s]} \times \text{hourly-rate}$  (Table 4.2) / 3600. Multiply by `epochsTotal` to obtain the optimiser’s energy term for a plan that runs the job to completion on the listed bundle.

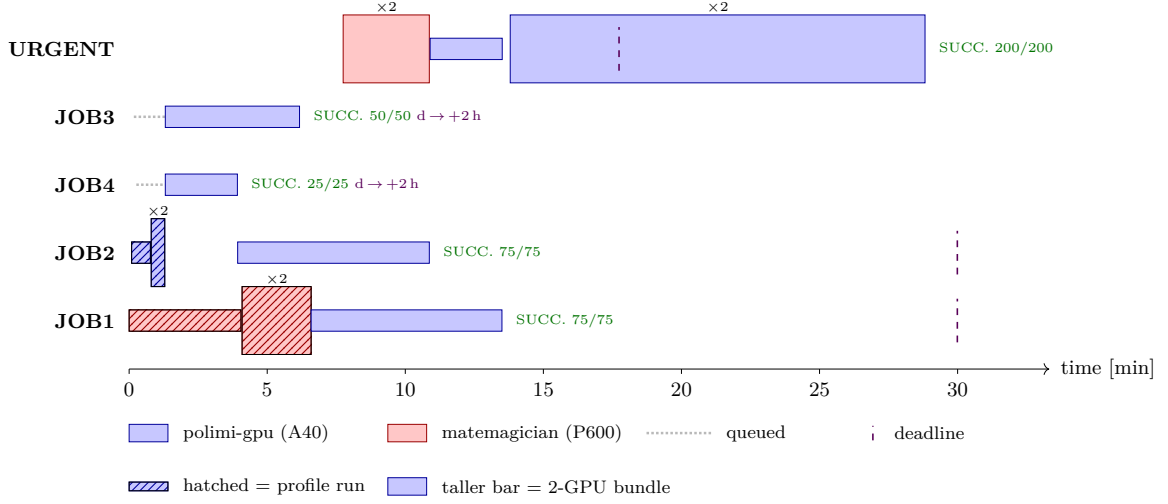
## 4.3 Scenario 1 — single type

We submit five `cnn_big` jobs (Table 4.6) with loose patient deadlines. The cluster is *under-subscribed* when URGENT arrives — both P600 slots free — so the horizon-myopic proxy parks it on the free P600×2 rather than displacing a patient for faster A40, and it later migrates onto A40 as the patients finish, never by eviction (Table 4.8). No bundle can meet URGENT’s +10-min deadline at 200 epochs regardless.

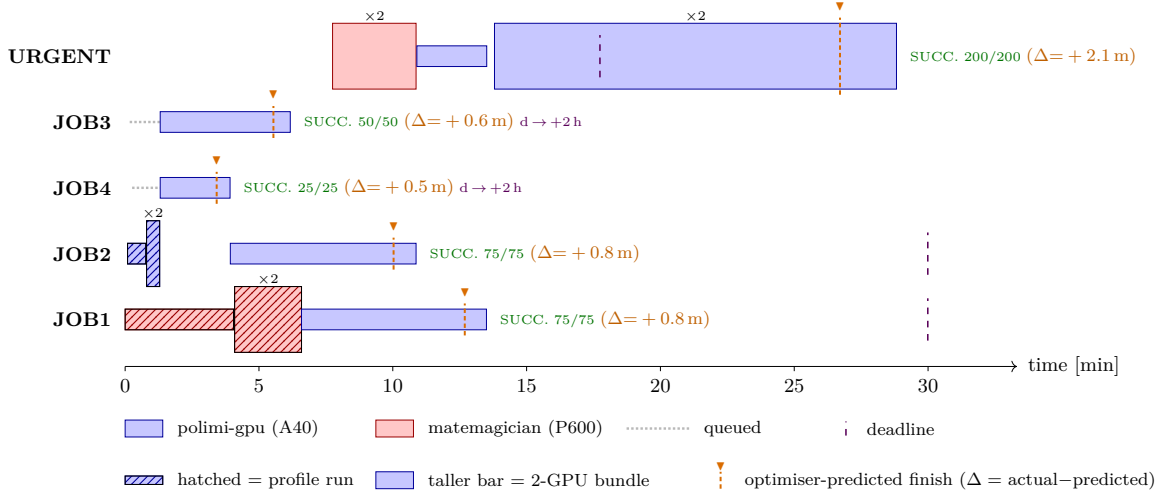


| Label   | Type    | Priority | Deadline | Epochs | Submitted at |
|---------|---------|----------|----------|--------|--------------|
| JOB1    | cnn_big | 4        | +30 min  | 75     | $t = 0$ s    |
| JOB2    | cnn_big | 4        | +30 min  | 75     | $t = 5$ s    |
| JOB3    | cnn_big | 2        | +2 h     | 50     | $t = 10$ s   |
| JOB4    | cnn_big | 1        | +2 h     | 25     | $t = 15$ s   |
| URGENT* | cnn_big | 5        | +10 min  | 200    | $t=7.5$ min  |

**Table 4.6:** Scenario 1 submissions.



**Figure 4.1:** Scenario 1 timeline (run 2026-06-25,  $t=0$  at first dispatch). URGENT lands on the free P600 $\times$ 2 (the whole **matemagician** node is idle; no patient is preempted), then migrates onto A40 — first A40 $\times$ 1, then A40 $\times$ 2 — as the two prio-4 patients finish. The proxy mechanic is unpacked in §4.3.2.



**Figure 4.2:** Scenario 1, optimiser-predicted finish vs. actual (run 2026-06-25, after the profiler-measurement fix). The orange  $\blacktriangledown$  marks where the optimiser predicted each job’s *completing* run would end — its ExecTime estimate `remaining_epochs` (from the `OPT REQ` that placed it)  $\times$  the steady-state per-epoch of its final bundle (Table 4.4), anchored at that placement’s start — against the bar’s true end;  $\Delta$  = actual–predicted. Every  $\Delta$  is now within a few minutes and slightly positive: the estimate counts only epoch compute, so it misses each segment’s one-off container start and checkpoint load. The four patients land at +0.5 to +0.8 min; URGENT’s larger +2.1 min is the cross-node resume cost of its P600→A40 migration.

| Bundle       | per-epoch [s] | GPUcost/ep [\$] | 50-ep [\$] | 200-ep [\$]   |
|--------------|---------------|-----------------|------------|---------------|
| A40×1        | 5.08          | 0.000423        | 0.0212     | <b>0.0847</b> |
| A40×2        | 4.55          | 0.000695        | 0.0348     | 0.1390        |
| QuadroP600×1 | 78.36         | 0.000653        | 0.0327     | 0.1306        |
| QuadroP600×2 | 46.66         | 0.000713        | 0.0356     | 0.1426        |

**Table 4.7:** Per-bundle data the optimiser sees for `cnn_big` in Scenario 1. Full-run (200-ep) energy is now cheapest on A40×1 (bold), not P600 — P600 is  $\sim 15\times$  slower, outweighing its  $\sim 10\times$  cheaper hourly rate — yet the proxy still picks P600 (Table 4.9): it bills scheduled jobs only to the horizon at the hourly rate, not to real completion.

### 4.3.1 Decision timeline

| $t$ [min]   | Job(s)        | Decision (why)                                                                                                                                                                                                                                                                                                       |
|-------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.0         | J1, J2        | J1 begins the P600 profile sweep; J2 the A40 sweep. “Profile-always” gate: neither is yet a standard candidate.                                                                                                                                                                                                      |
| 1.3         | J3, J4        | Once the type’s A40 cells are measured, prio-2 J3 (50 ep) and prio-1 J4 (25 ep) are placed standard on A40×1 each.                                                                                                                                                                                                   |
| 3.9         | J4, J2        | J4 ends naturally (25/25); J2’s sweep finishes, so OPT places prio-4 J2 on the freed A40×1 — no eviction.                                                                                                                                                                                                            |
| 6.2         | J3            | J3 ends naturally (50/50).                                                                                                                                                                                                                                                                                           |
| 6.6         | J1            | J1’s sweep finishes; OPT places prio-4 J1 on the freed A40×1. Cluster: J1, J2 on A40×1; <b>both P600 slots free</b> .                                                                                                                                                                                                |
| <b>7.7</b>  | <b>URGENT</b> | <b>Submitted (prio 5, +10 min, 200 ep). OPT places it on the free P600×2.</b> The node is idle, so no preemption; the horizon-myopic proxy reads URGENT’s tardiness as zero (logged <code>expected_tardiness=0</code> ) and prefers the cheap free P600 over displacing a patient for A40. Chosen-plan cost \$0.030. |
| <b>10.9</b> | <b>URGENT</b> | <b>J2 ends (75/75); OPT migrates URGENT P600×2 → A40×1</b> onto the freed slot ( <b>drop-preempt</b> empty — a migration, not an eviction).                                                                                                                                                                          |
| <b>13.5</b> | <b>URGENT</b> | <b>J1 ends (75/75); OPT migrates URGENT A40×1 → A40×2</b> , both A40 GPUs now free. Again no patient is displaced — both finished on their own.                                                                                                                                                                      |
| 17.7        | URGENT        | Deadline (violet tick on Fig.4.1); URGENT is ~40% through, already on A40×2.                                                                                                                                                                                                                                         |
| 28.8        | URGENT        | URGENT ends (200/200), ~11 min past deadline. $w_5 \times 678 \text{ s} \approx \$0.30$ tardiness incurred.                                                                                                                                                                                                          |

**Table 4.8:** Scenario 1 decision log (2026-06-25).

### 4.3.2 URGENT plan comparison

We report what the optimiser actually scored. At URGENT’s submission ( $t \approx 7.7$  min) the whole P600 node is free, and `proxy_function_min` ranks the plan that drops URGENT onto that free P600×2 as *cost-minimal* — the run’s OPT RESP logs its cost at \$0.030 — even though that bundle overruns the +10-min deadline by ~2.4 h. Table 4.9 shows why: the proxy charges every job only up to the horizon  $\Delta t = \text{first\_finish\_time}$  (the soonest finish in the plan), and in every cheap plan a prio-4 patient finishes first a few minutes out — so URGENT’s tardiness, read at that horizon, is exactly zero on every bundle except the one that makes URGENT itself finish first.

| Plan for URGENT           | ExecTime [s]           | meets +10 min? | in-proxy Tardy [s] | Proxy verdict                                                                                                                                                                                                                                                              |
|---------------------------|------------------------|----------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A40×2                     | 910 (15 min)           | no             | 310                | <code>get_best_setup</code> 's own pick (fastest, since no bundle is deadline-reachable), but RG rejects it: URGENT becomes the earliest finish (patients displaced to slow P600), stretching $\Delta t$ and finally exposing its own tardiness $\Rightarrow$ dearest plan |
| A40×1                     | 1 016 (17 min)         | no             | 0                  | tardiness ties the chosen plan, but the A40 hourly rate is 10× P600's, so over the horizon it scores dearer — never preferred for URGENT                                                                                                                                   |
| P600×1 (free)             | 15 672 (261 min)       | no             | 0                  | near-ties the chosen plan (also free, cheapest rate, tardiness masked); RG took the 2-GPU bundle of the same idle node                                                                                                                                                     |
| <b>P600×2 (free node)</b> | <b>9 332 (155 min)</b> | <b>no</b>      | <b>0</b>           | <b>✓ chosen (cost-minimal): the whole P600 node is idle, a prio-4 patient sets the <math>\sim 3</math> min horizon, so URGENT's 2.4-h overrun stays invisible and its GPU is billed for only <math>\sim 3</math> min of cheap P600</b>                                     |

**Table 4.9:** URGENT plan comparison, Scenario 1 — the values `proxy_function_min` actually scores at submission. ExecTime = 200× per-epoch (Table 4.4); no bundle reaches the +10-min deadline, so `get_best_setup` would take the fastest (A40×2), but the full-plan proxy overrides it. in-proxy Tardy =  $[t_0 + \Delta t - d]^+$  with  $\Delta t = \text{first\_finish\_time}$ : zero whenever a quick patient (not URGENT) sets  $\Delta t$ , positive only in the A40×2 plan where URGENT itself finishes first. The chosen plan's logged proxy cost is \$0.030.

Under a *deadline-aware* objective — one that scored each job's *real* finish time, as `method=PR`'s `proxy_function_max` does — this ranking inverts: A40×2 becomes best while the slow free-P600 bundles fall to the bottom. Table 4.10 gives that counterfactual full-run yardstick; it is *not* what the reference `method=RG` deployment minimises — by it, the run's P600×2 choice costs  $\sim 13\times$  the ideal A40×2.

| Plan for UR-<br>GENT | ExecTime [s] | Tardy [s] | GPUCost [\$] | NodeCost [\$] | TardCost [\$] | <b>Total [\$]</b> | rank            |
|----------------------|--------------|-----------|--------------|---------------|---------------|-------------------|-----------------|
| A40×2                | 911          | 311       | 0.139        | 0.101         | 0.138         | <b>0.378</b>      | <b>1 (best)</b> |
| A40×1                | 1 016        | 416       | 0.085        | 0.113         | 0.185         | 0.383             | 2               |
| P600×2               | 9 332        | 8 732     | 0.143        | 1.037         | 3.877         | 5.057             | 3               |
| (run's choice)       |              |           |              |               |               |                   |                 |
| P600×1               | 15 672       | 15 072    | 0.131        | 1.741         | 6.692         | 8.564             | 4 (worst)       |

**Table 4.10:** *Counterfactual* idealised full-run cost — *not* the deployed objective. Same terms as `proxy_function_min` but with tardiness, GPU, and node cost measured at each job's *real* completion instead of at  $\Delta t$ . Tardy =  $\max(\text{ExecTime} - 600, 0)$ ; TardCost = Tardy  $\times w_5$  ( $w_5 = 0.000444$  \$/s); NodeCost  $\approx (\text{cost}_{\text{mate}} + \text{cost}_{\text{polimi}}) \times \text{ExecTime} / 3600$ . This is how a deadline-aware proxy *would* rank the plans — A40×2 best; the run took P600×2 (3rd of 4,  $\sim 13\times$  the ideal), because that minimised the horizon-myopic proxy actually deployed (Table 4.9), not this yardstick.

As the patients finish on their own (JOB2 at  $t \approx 10.9$ , JOB1 at  $t \approx 13.5$ ), the optimiser migrates URGENT onto A40 — first A40×1, then A40×2 — without displacing anyone; but the slow-P600 start and staged migration leave it  $\sim 11$  min late. (URGENT cannot meet its +10-min deadline on *any* bundle: even A40×2 needs  $\sim 15$  min for 200 epochs.)

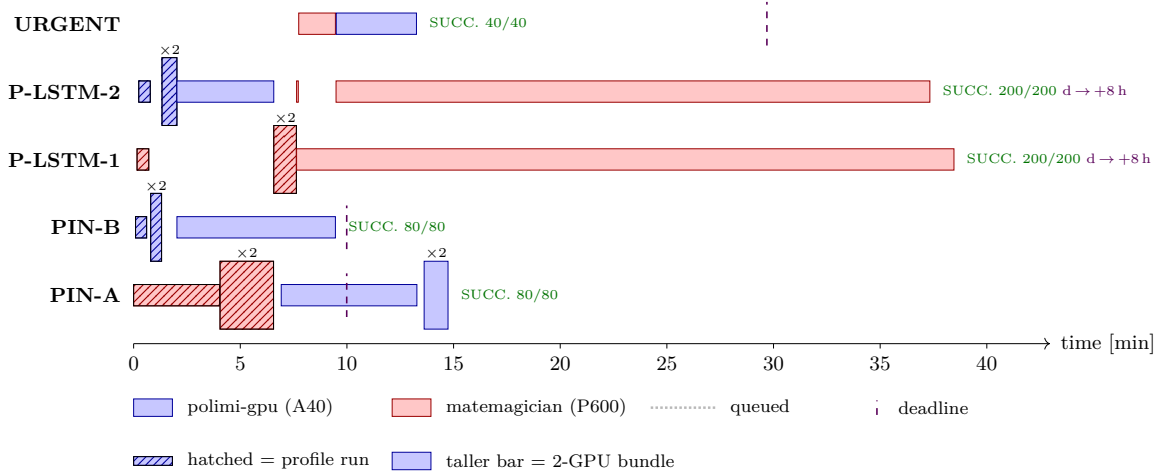
method=PR (with `proxy_function_max`, deadline-aware per job) would have seen URGENT’s true finish time from the first round and rushed it onto A40×2 immediately — preempting both patients up front and finishing  $\sim 5$  min late instead of  $\sim 11$  — but we keep method=RG as the reference deployment.

#### 4.4 Scenario 2 — two types, forced 2-GPU placement

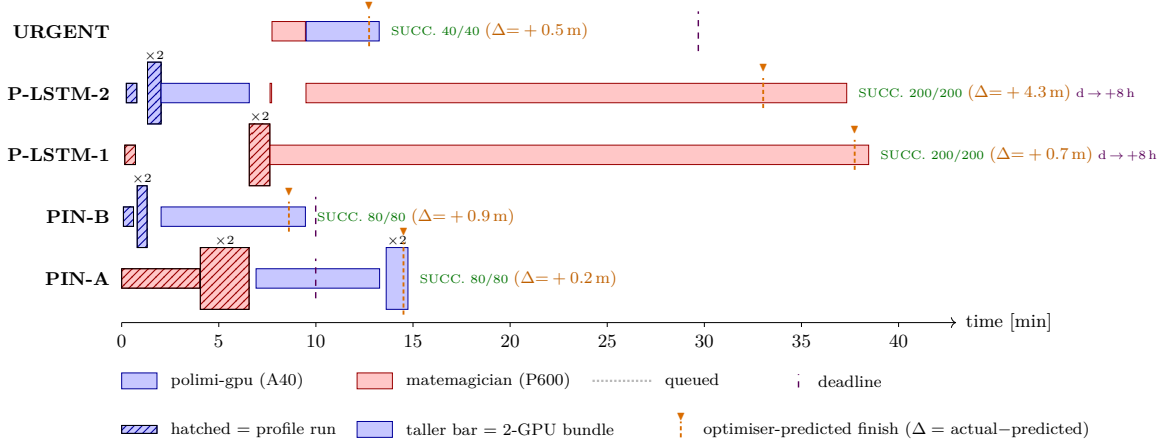
Scenario 2 introduces a second job type and *fills* the cluster: two prio-5 `cnn_big` pins on A40, two slack prio-1 `lstm-small` on P600. On the true per-epoch costs no *preemptible* bundle meets URGENT’s +22-min deadline (A40 would, but the equal-priority pins block it; P600×2 needs  $\sim 31$  min, P600×1  $\sim 52$ ), so — as in Scenario 1 — horizon-myopia takes the cheapest preemptible bundle, P600×1, evicting one slack patient; URGENT meets its deadline only by later migrating onto freed A40 (Table 4.12 and the decision log). Both scenarios thus show the same myopia.

| Label    | Type                          | Priority | Deadline | Epochs |
|----------|-------------------------------|----------|----------|--------|
| PIN-A    | B ( <code>cnn_big</code> )    | 5        | +10 min  | 80     |
| PIN-B    | B ( <code>cnn_big</code> )    | 5        | +10 min  | 80     |
| P-LSTM-1 | S ( <code>lstm-small</code> ) | 1        | +8 h     | 200    |
| P-LSTM-2 | S ( <code>lstm-small</code> ) | 1        | +8 h     | 200    |
| URGENT*  | B ( <code>cnn_big</code> )    | 5        | +22 min  | 40     |

**Table 4.11:** Scenario 2 submissions (2026-06-25). On the true per-epoch costs, URGENT’s +22-min deadline is met only by A40 (3–4 min), held by the equal-priority (prio-5) pins URGENT cannot preempt; no P600 bundle reaches it (P600×2  $\sim 31$  min, P600×1  $\sim 52$  min).



**Figure 4.3:** Scenario 2 timeline (run 2026-06-25,  $t=0$  at first dispatch). The cluster is full and no P600 bundle meets URGENT’s +22-min deadline, so the horizon-myopic plan takes the cheapest one — P600×1, preempting a *single* prio-1 `lstm-small` — and leaves the equal-priority A40 pins alone. URGENT then migrates to A40×1 the moment PIN-B ends, where it meets the deadline with time to spare; the evicted patient resumes on P600.



**Figure 4.4:** Scenario 2, optimiser-predicted finish ( $\blacktriangledown$ ) vs. actual (run 2026-06-25), with the convention of Fig. 4.2. After the profiler fix the predictions are tight — most jobs land within  $\sim 1$  min (the startup-and-checkpoint floor). The outlier is P-LSTM-2 at  $+4.3$  min: it was preempted for URGENT and resumed sharing *matemagician*, so its final run hit co-location contention ( $\sim 10.6$  s/epoch vs. the solo profile’s  $\sim 9.2$  s) that a solo profile cannot foresee.

#### 4.4.1 URGENT plan comparison

| Plan                                                  | ExecTime      | meets +22 min? | GPUCost [\$]  | Decision                                                                                                                                                                                                      |
|-------------------------------------------------------|---------------|----------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A40 $\times$ 1 (evict 1 prio-5 pin)                   | 3.4 min       | yes            | 0.0171        | off-limits: would evict an equal-priority pin, which URGENT cannot preempt                                                                                                                                    |
| A40 $\times$ 2 (evict 2 prio-5 pins)                  | 3.3 min       | yes            | 0.0298        | off-limits: same, for <i>both</i> equal-priority pins                                                                                                                                                         |
| P600 $\times$ 2 (evict 2 prio-1 lstm)                 | 31 min        | no             | 0.0285        | rejected: dearer 2-GPU rate than P600 $\times$ 1 over the horizon, and evicts a second patient, for no proxy gain (tardiness masked either way)                                                               |
| <b>P600<math>\times</math>1 (evict 1 prio-1 lstm)</b> | <b>52 min</b> | <b>no</b>      | <b>0.0261</b> | <b>✓ chosen:</b> the only deadline-meeting bundles (A40) are off-limits, so RG takes the cheapest <i>preemptible</i> one; horizon-myopia reads URGENT’s tardiness as zero, so the 52-min overrun is invisible |

**Table 4.12:** URGENT plan comparison, Scenario 2 (2026-06-25). ExecTime =  $40 \times$  per-epoch (Table 4.4); GPUCost is the full-run energy. Both deadline-meeting bundles are A40, held by equal-priority pins URGENT cannot preempt; among the *preemptible* (P600) bundles neither meets the +22-min deadline, so there is no deadline-reachable option for *get\_best\_setup* and RG falls back to plain cost minimisation — with tardiness masked by horizon-myopia, that is simply the cheapest P600 bundle, P600 $\times$ 1. URGENT nonetheless *meets* its deadline: when PIN-B finishes it migrates onto the freed A40 $\times$ 1 (decision log below). The node term is omitted: the 200-epoch lstm-small patients set the  $\sim 38$ -min makespan, so per-node cost ( $\approx (\text{cost}_{\text{mate}} + \text{cost}_{\text{polimi}}) \times 38 \text{ min} \approx \$0.25$ ) is near-identical across URGENT placements and cannot change the choice.

#### 4.4.2 Decision timeline

| $t$ [min]  | Job(s)                  | Decision (why)                                                                                                                                                                                                                                                                                        |
|------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.0–6.6    | all four                | Profile sweep over both types' eight (type, GPU-bundle) cells. PIN-A carries the P600 sweep; PIN-B + P-LSTM-2 carry the A40 sweep; P-LSTM-1 atomic-claims the remaining cells.                                                                                                                        |
| 2.0        | PIN-B, P-LSTM-2         | Both finish their profile budgets; OPT places PIN-B (prio 5) and P-LSTM-2 (prio 1) on A40×1 each (only profiled bundles available).                                                                                                                                                                   |
| 6.9        | PIN-A, P-LSTM-2         | PIN-A's sweep finishes; OPT prefers prio-5 PIN-A over prio-1 P-LSTM-2 on A40 $\Rightarrow$ <i>drop-preempts</i> P-LSTM-2; PIN-A on A40×1.                                                                                                                                                             |
| 7.6        | P-LSTM-1, P-LSTM-2      | OPT places P-LSTM-1 and the just-evicted P-LSTM-2 on matemagician P600×1 each. Cluster now <b>full</b> : A40 = two prio-5 pins, P600 = two prio-1 lstm.                                                                                                                                               |
| <b>7.7</b> | <b>URGENT</b>           | <b>Submitted (prio 5, +22 min, 40 ep). No P600 bundle meets the deadline, so the horizon-myopic plan takes the cheapest one — P600×1 — drop-preempting a <i>single</i> prio-1 lstm (P-LSTM-2).</b> A40 is held by the equal-priority pins; chosen-plan cost \$0.016, URGENT's tardiness read as zero. |
| <b>9.5</b> | <b>URGENT, P-LSTM-2</b> | <b>PIN-B ends (80/80); OPT migrates URGENT P600×1 <math>\rightarrow</math> A40×1 (faster, now free); the evicted P-LSTM-2 resumes on P600×1.</b>                                                                                                                                                      |
| 13.3       | URGENT                  | URGENT ends (40/40) on A40×1, $\sim$ 16 min <i>inside</i> its +22-min deadline.                                                                                                                                                                                                                       |
| 14.7       | PIN-A                   | PIN-A ends (80/80); the second A40 frees.                                                                                                                                                                                                                                                             |
| $\sim$ 38  | P-LSTMs                 | Both lstm-small patients finish on P600×1 (their +8-h deadlines are far off).                                                                                                                                                                                                                         |

Table 4.13: Scenario 2 decision log (2026-06-25).

#### 4.5 Reproducibility

All figures and tables derive from the 2026-06-25 run snapshots committed under `documentation/report/e2e_runs/2026-06-25/{s1,s2}/`:

- **Committed artifacts:** `api.log`, `jobs.json`, `profiles_*.json`, `chart_s{1,2}{, _predicted}.tex`.
- **Per-bundle costs:** `profiling_results.duration_seconds`  $\times$  the hourly rates (Table 4.2). These runs use the corrected profiler — the worker records the runtime's own per-epoch compute time (warmup excluded), not log-line arrival intervals (`worker/profiling.py`, `execution.py`).
- **Charts:** auto-generated from the API log by `generate_chart_tex.py` (predicted overlay: `generate_chart_tex_predicted.py`); every bar is a real log event.
- **Drivers:** `infra/e2e_scenario.sh`, `infra/e2e_scenario_2types.sh`; slots at `GET /admin/slots`; API logs via `docker logs ijm-api` (stack up by `infra/ijm up`).

## 5. Conclusion

### 5.1 Summary of contributions

IJM extends the ANDREAS prototype from a single-host scheduler into a distributed control plane: one FastAPI service drives per-node workers over SSH-tunnelled HTTP, and an external GPUspb optimiser is consumed as a peer service rather than embedded in the scheduler. On top of that topology the report names three substantive contributions. A *profile-always* policy keyed by the unique index on (`job_id`, `gpu_config`) — so that profile measurements are atomically shared across instances of the same job type. *Cross-version checkpoint resume* between PyTorch 1.5.1 (CUDA 10.1, `matemagician`) and PyTorch 2.6 (CUDA 12.4, `polimi-gpu`), enabling an in-flight job to migrate between heterogeneous GPU generations without losing progress (§2.7). And an empirical characterisation of GPUspb’s `method=RG` proxy (Chapter 4). Scenario 1 isolates its *horizon-myopia* — tardiness scored at the next-event horizon, so an urgent job is parked on a slow but *free* bundle whose deadline overrun the proxy cannot see — while Scenario 2 shows the same myopia on a *full* cluster: with no preemptible bundle fast enough to meet the urgent job’s deadline, the stateless cost minimisation drops the cheapest slack work and parks the arrival on the cheapest *slow* bundle, and the deadline is recovered only when a pin finishes and the job migrates onto freed fast hardware. The priority-aware *outcome* — evicting slack prio-1 work but never an equal-or-higher-priority job, and still hitting the deadline — is emergent from the unscheduled-job penalty (multiplier  $\lambda=100$ ) plus IJM’s migrate-on-free-slot behaviour, not a preemption or deadline policy in the optimiser.

### 5.2 Limitations

The reference deployment uses GPUspb’s `method=RG`, whose proxy is horizon-myopic; the deadline-aware `method=PR` variant exists upstream but is not the default and has not been validated end to end against the IJM control plane. The report contains no baseline-algorithm comparison — IJM’s behaviour is documented against the idealised full-run yardstick, but not yet against FIFO, EDF, or priority-only baselines on the same workloads. Worker-side coverage is almost entirely end-to-end — `worker/tests/` holds only a regression test for the per-epoch profiler measurement; only two GPU generations (A40 and QuadroP600) have been validated; and the cluster used throughout Chapter 4 is two nodes with four GPU slots in total, so the scaling properties of the four-phase pass at tens of nodes are an open question.

### 5.3 Future work

Two follow-ups have concrete entry points in the codebase. First, re-run Scenario 1 with `OPTIMIZER_METHOD=PR`: `proxy_function_max` is per-job deadline-aware, so the prediction from §4.3.2 is that URGENT is rushed onto the fastest hardware (preempting the prio-4 patients) from round 1, rather than parking on the slow free P600 first and finishing  $\sim 11$  min late. Second, extend the two-node evaluation to a larger cluster once a third GPU class is available; this is where the profile-claim’s atomicity and the optimiser-call latency (currently a 30 s timeout) will be stressed.



# Bibliography

- [AF<sup>+</sup>21] Danilo Ardagna, Federica Filippini, et al. ANDREAS – artificial intelligence training scheduler for accelerated clusters. Technical report, Politecnico di Milano / TETRAMAX, 2021. Horizon 2020 project deliverables D1–D3.